▶ 작업 보고서: 게시글 검색 기능 (1단계: SSR 기반 기본 검색)

- **작업 일시**: 2026-03-24
- **진행 단계**: 완료

1. 🌊 전체 작업 흐름 (Workflow)

사용자가 검색어를 입력하고 버튼을 누르면, 서버가 데이터를 필터링하여 다시 전체 페이지를 그려주는(SSR) 흐름입니다.

```
1. [사용자] 검색어 입력 (예: "안녕") 및 [검색] 버튼 클릭
2. [브라우저] GET /?keyword=안녕 요청 전송
3. [컨트롤러] keyword="안녕" 파라미터 수신 -> 서비스로 전달
4. [서비스]
   - 키워드가 있으면: Repository.findAllByKeyword() 호출
   - 키워드가 없으면: Repository.findAll() 호출
5. [레포지토리] SQL 실행 (WHERE title LIKE %안녕% OR content LIKE %안녕%)
6. [머스테치] 검색어 유지 + 필터링된 목록으로 화면 렌더링
```

2. 🧩 변경된 모든 코드 포함

1) BoardRepository.java (데이터 조회 규칙 추가)

```java
// 키워드를 포함하는 게시글을 페이징하여 조회하는 쿼리 추가
@Query("SELECT b FROM Board b WHERE b.title LIKE %:keyword% OR b.content LIKE
%:keyword% ORDER BY b.id DESC LIMIT :limit OFFSET :offset")
List<Board> findAllByKeyword(@Param("keyword") String keyword, @Param("limit") int
limit, @Param("offset") int offset);

// 검색 결과의 전체 개수를 세는 쿼리 (페이징 계산용)
@Query("SELECT COUNT(b) FROM Board b WHERE b.title LIKE %:keyword% OR b.content
LIKE %:keyword%")
Long countByKeyword(@Param("keyword") String keyword);
```

2) BoardService.java (비즈니스 로직 분기)

```java
public BoardResponse.ListDTO 게시글목록보기(int page, String keyword) {
    int limit = 3;
    int offset = page * limit;

    List<Board> boardList;
    Long totalCount;
```

```java
    // 검색어 유무에 따라 다른 레포지토리 메서드 호출
    if (keyword == null || keyword.isBlank()) {
        boardList = boardRepository.findAll(limit, offset);
        totalCount = boardRepository.countAll();
    } else {
        boardList = boardRepository.findAllByKeyword(keyword, limit, offset);
        totalCount = boardRepository.countByKeyword(keyword);
    }

    // 결과가 없으면 404 예외 발생 (디시즌 준수)
    if (boardList.isEmpty() && page > 0) {
        throw new Exception404("더 이상 게시글이 없습니다.");
    }

    return new BoardResponse.ListDTO(boardList, page, totalCount, limit, keyword);
}
```

3) BoardController.java (문지기 역할)

```java
@GetMapping("/")
public String list(@RequestParam(defaultValue = "1", name = "page") int page,
                   @RequestParam(defaultValue = "", name = "keyword") String
keyword,
                   Model model) {
    // 1페이지 미만 요청 시 리다이렉트 (검색어 유지)
    if (page < 1) {
        return "redirect:/?page=1&keyword=" + keyword;
    }

    // 서비스 호출 시 키워드 함께 전달
    var responseDTO = boardService.게시글목록보기(page - 1, keyword);
    model.addAttribute("model", responseDTO);

    return "board/list";
}
```

4) list.mustache (UI 및 링크)

```html
<!-- 검색 폼 추가 -->
<form action="/" method="get" class="d-flex mr-3">
    <input class="form-control form-control-sm mr-2" type="text" name="keyword"
placeholder="검색어 입력" value="{{model.keyword}}">
    <button class="btn btn-sm btn-outline-primary" type="submit">검색</button>
</form>

<!-- 페이징 링크에 검색어 유지 -->
<a class="page-link" href="/?page={{model.prevPage}}&keyword={{model.keyword}}">이
전</a>
```

3. 🎈 상세비유 쉬운 예시 (Easy Analogy)

"이번 작업은 **도서관의 사서에게 검색 조건을 말하는 것**과 같습니다. 이전에는 사서에게 '그냥 책 다 보여주세요(전체 목록)'라고만 했다면, 이제는 '제목이나 내용에 "컴퓨터"가 들어간 책만 골라서 보여주세요(검색)'라고 구체적으로 요청하는 기능을 만든 것입니다. 사서는 그 조건에 맞는 책들만 골라서(DB 필터링) 책수레(Model)에 담아 독자(브라우저)에게 가져다줍니다."

4. 📚 기술 딥다이브 (Technical Deep-dive)

- **JPQL LIKE 연산자**: SQL의 `LIKE`와 동일하게 문자열 패턴 매칭을 수행합니다. `%:keyword%` 형식을 통해 키워드가 앞, 뒤, 중간 어디에 있든 찾아낼 수 있습니다.
- **Query Parameter 유지**: SSR 방식에서는 페이지 이동 시 상태가 초기화되므로, `<a>` 태그의 `href`에 현재 검색어(`keyword`)를 계속 붙여서 넘겨주어야 검색 결과 내에서 페이징이 정상 동작합니다.
- **DefaultValue 처리**: 사용자가 키워드 없이 접속할 경우를 대비해 컨트롤러에서 `@RequestParam(defaultValue = "")`를 설정하여 `null` 에러를 방지했습니다.